Report 2: Routy

Rodrigo Perestrelo, Lennard Last
ID2201 Distributed Systems, Basic Course

September 16, 2026

1 Introduction

The objective of this assignment is to design, implement, and empirically evaluate a decentralized link-state routing protocol. This protocol requires every router to independently construct an identical global graph of the network topology and locally compute optimal next-hop forwarding paths using Dijkstra's algorithm.

2 What Was Difficult and How It Was Solved

2.1 Implementing Dijkstra's Algorithm and Routing Table Construction

The Problem: Implementing Dijkstra's algorithm in a purely functional language required iteratively relaxing paths while keeping track of the direct next-hop gateway for every destination.

The Solution: In `update/4`, we check whether a node already exists in the sorted list via `entry/2`. If absent (returning `0`), it has already been settled and is safely discarded. If present with a greater distance, it is replaced with the shorter path and the corresponding gateway:

```
update(Node, N, Gateway, Sorted) ->
    case entry(Node, Sorted) of
        0 -> Sorted;
        Length when Length > N ->
            replace(Node, N, Gateway, Sorted);
        Length when Length =< N ->
            Sorted
    end.
```

During table generation in `iterate/3`, the node with the minimal distance is extracted from the sorted list, committed to the final routing table as `{Dest, Gateway}`, and its adjacent links from `Map` are relaxed using `update/4` before recursing.

2.2 Preventing Broadcast Storms and Stale LSAs in Cyclic Networks

The Problem: Link-state advertisements flooded over cyclic topologies will loop indefinitely if routers lack memory of forwarded packets, causing broadcast storms and packet duplication.

The Solution: In `hist.erl`, each router tags advertisements with an incrementing integer sequence counter N. A received message from router $Node$ with sequence number R is discarded as `old` if $R \leq$ Counter. Furthermore, by initializing a router's own identity as $\{Name, \text{inf}\}$, any self-generated message that cycles back through the network is rejected:

```
update(Node, N, History) ->
    case lists:keyfind(Node, 1, History) of
        false -> {new, [{Node, N} | History]};
        {Node, Counter} when Counter < N ->
```

```
            NewHist = lists:keydelete(Node, 1, History),
            {new, [{Node, N} | NewHist]};
        {Node, Counter} when Counter >= N -> old
    end.
```

2.3 Solved Improvement: Autonomous Event-Driven Convergence

The Problem: In the base specification, link-state flooding (`broadcast`) and path recomputation (`update`) required manual invocation via external console messages whenever a network event occurred.

The Solution: We eliminated manual steps by integrating autonomous reactive flooding directly into `routy.erl`. When a router traps a `'DOWN'` process exit, or executes an `add` or `remove` interface mutation, it increments sequence counter N, immediately broadcasts its updated interface list to all direct peers via `intf:broadcast/2`, and recomputes its table. Furthermore, upon receiving a novel link-state advertisement (`{links, ...}`), routers automatically update their graph and recalculate their Dijkstra table on the fly:

```
{'DOWN', Ref, process, _, _} ->
    {ok, Down} = intf:name(Ref, Intf),
    io:format("~w: exit recived from ~w~n", [Name, Down]),
    Intf1 = intf:remove(Down, Intf),
    Message = {links, Name, N, intf:list(Intf1)},
    intf:broadcast(Message, Intf1),
    Table1 = dijkstra:table(intf:list(Intf1), Map),
    router(Name, N+1, Hist, Intf1, Table1, Map);
{links, Node, R, Links} ->
    case hist:update(Node, R, Hist) of
        {new, Hist1} ->
            intf:broadcast({links, Node, R, Links}, Intf),
            Map1 = map:update(Node, Links, Map),
            Table1 = dijkstra:table(intf:list(Intf), Map1),
            router(Name, N, Hist1, Intf, Table1, Map1);
        old ->
            router(Name, N, Hist, Intf, Table, Map)
    end;
```

3 Remaining Potential Improvements to Routy

1. **Weighted Edge Costs:** The protocol currently assumes unit link costs (hop count). Incorporating latency or bandwidth metrics into `map.erl` would allow Dijkstra to select throughput-optimal paths across diverse physical connections.

2. **LSA Garbage Collection:** The history tracking list in `hist.erl` grows indefinitely as new nodes are discovered. Implementing a time-to-live (TTL) on history records would prevent unbounded memory growth in long-running networks.

4 Automated Testing and Bonus Evaluation: The World

To evaluate Routy's end-to-end performance and autonomous fault recovery, we implemented a 14-node European network harness (`world_test.erl`) interconnecting major transit centers across Scandinavia, Central Europe, and the Iberian Peninsula.



Figure 1: Topology of the 14-node network harness. Multi-homed connections provide redundancy across the Iberian and Central European corridors.

4.1 Can we send messages across distant nodes in the topology?

Yes. Once initial interface advertisements flooded through the network and Dijkstra tables converged, multi-hop routing succeeded across the continent from Stockholm to Lisbon:

```
1> world_test:route_message(stockholm, lisbon, ola).
routing message:  "ola"
from:  stockholm → to:  lisbon
stockholm:  routing message ("ola")
malmo:  routing message ("ola")
hamburg:  routing message ("ola")
frankfurt:  routing message ("ola")
paris:  routing message ("ola")
barcelona:  routing message ("ola")
madrid:  routing message ("ola")
lisbon:  received message "ola"
```

The computed primary shortest path traverses 7 intermediate hops:

stockholm → malmo → hamburg → frankfurt → paris → barcelona → madrid → lisbon

4.2 Will routing functionality still work if we stop routers or close nodes?

Yes, provided an alternate physical path exists in the topology.

We halted the critical Iberian junction router Barcelona (`r10`) using `world_test:kill_router(barcelona)`. Its monitored peers (`paris`, `madrid`, and `toulouse`) caught the termination exit immediately:

```
paris:  exit recived from barcelona
stop
madrid:  exit recived from barcelona
toulouse:  exit recived from barcelona
```

Thanks to our autonomous re-convergence enhancement, the surviving neighbors automatically broadcast their updated interface lists and recomputed their tables without manual console commands. Dispatching another message from Stockholm to Lisbon succeeded, dynamically routing around the dead Barcelona node via Toulouse:

```
3> world_test:route_message(stockholm, lisbon, ola).
routing message:  "ola"
from:  stockholm → to:  lisbon
stockholm:  routing message ("ola")
malmo:  routing message ("ola")
hamburg:  routing message ("ola")
frankfurt:  routing message ("ola")
paris:  routing message ("ola")
toulouse:  routing message ("ola")
madrid:  routing message ("ola")
lisbon:  received message "ola"
```

4.3 How long between a disabled network card and the delivery of a 'DOWN' message?

The detection latency depends directly on the failure mechanism:

- **Router Process Crash or Shell Exit (`q().`):** Takes < 2 **ms** to < 15 **ms**. When a process dies or an OS terminates the VM, the operating system kernel closes open TCP sockets immediately with a `FIN` or `RST` packet. The remote Erlang runtime traps this socket closure instantly.
- **Disabled Network Interface / Pulled Cable:** Takes **45 to 60 seconds**. When a physical link is severed, no TCP teardown packets are transmitted. Surviving nodes cannot distinguish an idle connection from a dead interface. The runtime must wait for Erlang's distribution heartbeat timer (`net_ticktime`, default 60 seconds) to expire before flagging the node as unreachable and dispatching the `'DOWN'` message.

5 Conclusions

This laboratory demonstrated the separation between the control plane (graph flooding and shortest-path computation) and the data plane (packet forwarding). Erlang's lightweight actor model, process monitors, and pattern matching provided clean abstractions for implementing link-state protocols. Integrating autonomous event-driven flooding eliminated the need for manual intervention, enabling self-healing network behavior upon node failure.